



# Advanced Python programming using AI tools

---

## Lecture 8: Unit testing

Lecturer: Dr. Havana Rika

# Today

---

1. Unit Testing
2. pytest & Prompt Engineering
3. GitHub Copilot



# Code Correctness

---

**Goal:** write code according to the requirements,

i.e. given some **input** → need to get the correct **output**.

+ need to check all extreme cases!

Checking code correctness via the **main** function likely means **manually** running a program from its entry point (main function) with **various inputs** to verify the **outputs** are **correct**, a method generally known as ad hoc testing or desk checking.

While simple, this is **not** the most **effective** or reliable method for ensuring full correctness.

```
def add_numbers(a, b):  
    return a + b  
  
def main():  
    # --- Test Case 1: Positive numbers ---  
    a1, b1, ex1 = 5, 10, 15  
    res1 = add_numbers(a1, b1)  
    if res1 == ex1:  
        print(f"Test Case 1 (Positive): PASSED. Expected {ex1}, Got {res1}")  
    else:  
        print(f"Test Case 1 (Positive): FAILED. Expected {ex1}, Got {res1}")  
  
    # --- Test Case 2: Negative numbers ---  
    a2, b2, ex2 = -1, -5, -6  
    res2 = add_numbers(a2, b2)  
  
    if res2 == ex2:  
        print(f"Test Case 2 (Negative): PASSED. Expected {ex2}, Got {res2}")  
    else:  
        print(f"Test Case 2 (Negative): FAILED. Expected {ex2}, Got {res2}")  
  
    # --- Test Case 3: Zero ---  
    a3, b3, ex3 = 0, 100, 100  
    res3 = add_numbers(a3, b3)  
  
    if res3 == ex3:  
        print(f"Test Case 3 (Zero): PASSED. Expected {ex3}, Got {res3}")  
    else:  
        print(f"Test Case 3 (Zero): FAILED. Expected {ex3}, Got {res3}")  
  
if __name__ == "__main__":  
    main()
```

```
Test Case 1 (Positive): PASSED. Expected 15, Got 15  
Test Case 2 (Negative): PASSED. Expected -6, Got -6  
Test Case 3 (Zero): PASSED. Expected 100, Got 100
```

# Unit Testing

---

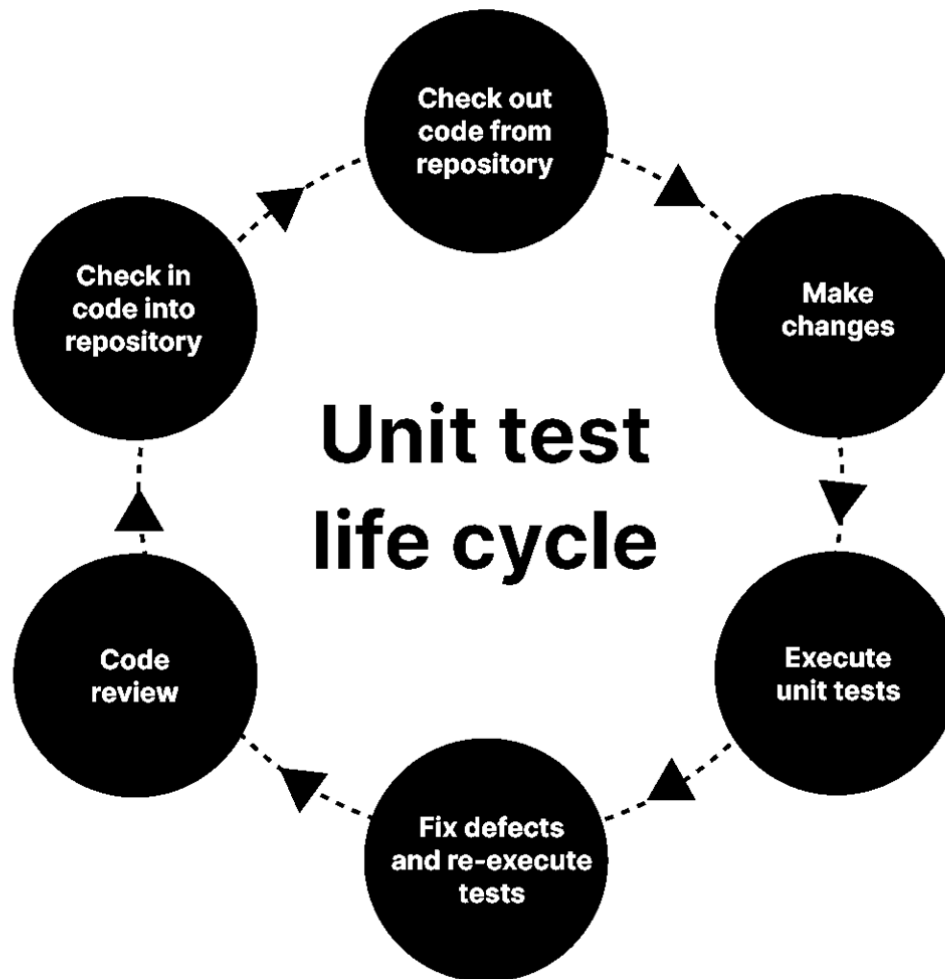
Unit tests are the most **effective** way to validate **small, isolated** pieces of **code** (units, such as functions or methods). You write specific test cases that provide a set of **inputs** and assert that the **output** is the expected result.

**Benefits:** Ensures code behaves as expected, prevents new changes from breaking existing functionality (regressions), and provides clear documentation of what the code is supposed to do.

**Tools:** Use testing frameworks specific to your programming language (e.g., pytest for Python, JUnit for Java, etc).

# Unit Testing

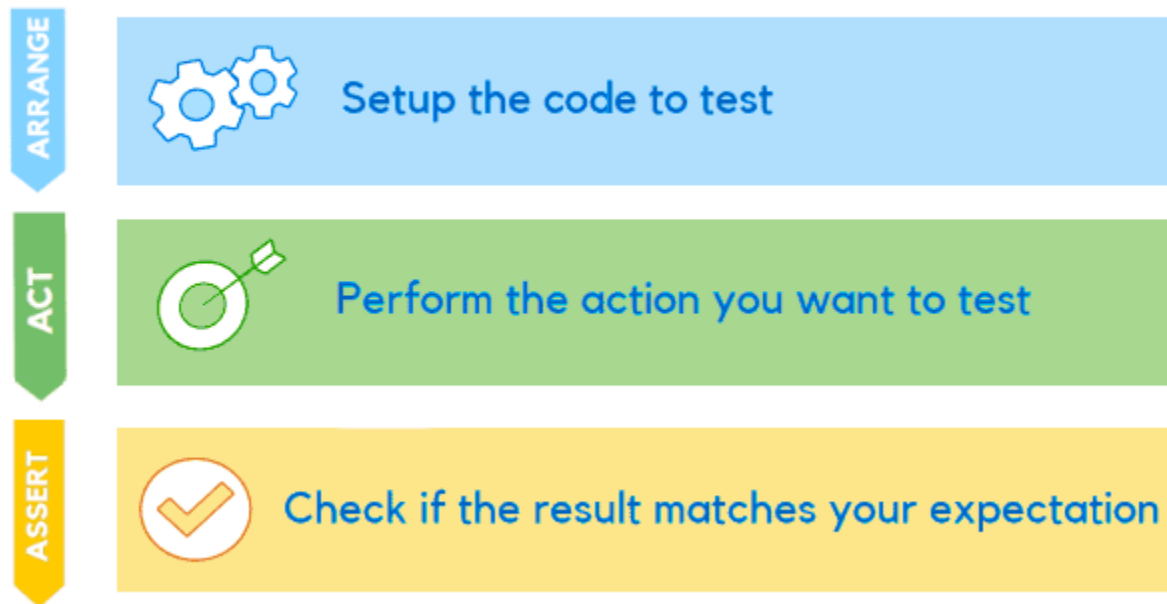
---



# The AAA Testing Pattern

---

## Arrange-Act-Assert (AAA)



<https://semaphore.io/blog/aaa-pattern-test-automation>

# The AAA Testing Pattern

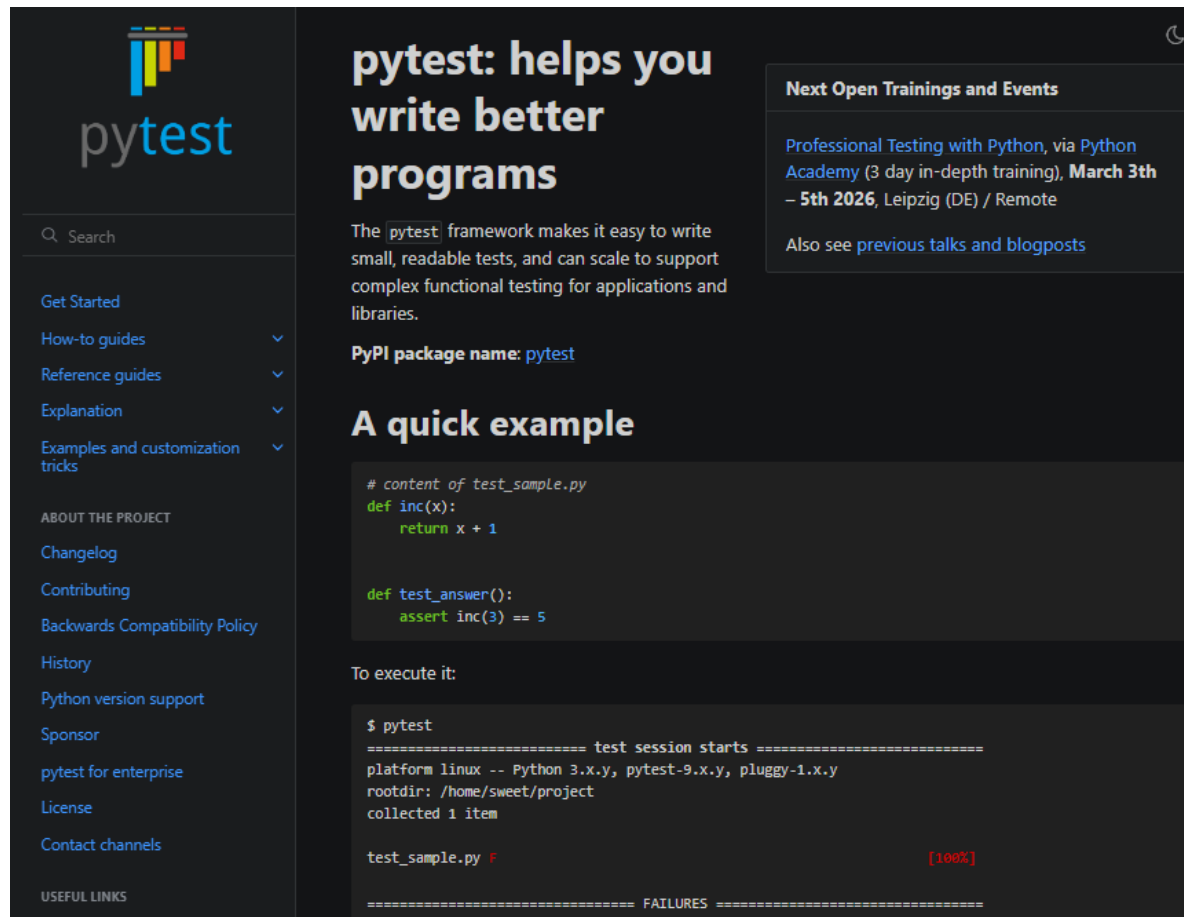
---

The Arrange-Act-Assert pattern, also known as the AAA or 3A pattern, is a widely recognized approach to structuring tests. It was originally proposed by [Bill Wake in 2001](#) and then mentioned in Kent Beck's influential book "[Test Driven Development: By Example](#)" in 2002.

The AAA pattern promotes clarity by recommending that tests be structured into three distinct phases:

- I. **Arrange** everything necessary to perform the test.
- II. **Act** on the target code to be tested by executing it.
- III. **Assert** the expected outcomes.

# PyTest



The screenshot shows the PyTest website homepage. On the left is a dark sidebar with the PyTest logo (a stylized 'P' made of colored bars) and the word 'pytest' in white. Below the logo is a search bar and a list of navigation links: 'Get Started', 'How-to guides', 'Reference guides', 'Explanation', 'Examples and customization tricks', 'ABOUT THE PROJECT', 'Changelog', 'Contributing', 'Backwards Compatibility Policy', 'History', 'Python version support', 'Sponsor', 'pytest for enterprise', 'License', 'Contact channels', and 'USEFUL LINKS'. The main content area has a dark background. At the top, it says 'pytest: helps you write better programs'. Below this is a paragraph explaining that the pytest framework makes it easy to write small, readable tests and can scale to support complex functional testing. To the right of this paragraph is a box titled 'Next Open Trainings and Events' containing information about a 'Professional Testing with Python, via Python Academy' training on March 3th-5th 2026 in Leipzig (DE) / Remote, with a link to 'previous talks and blogposts'. Below the paragraph is a section titled 'A quick example' showing a code snippet for a test file. The code defines a function 'inc(x)' that returns x + 1, and a test function 'test\_answer()' that asserts 'inc(3) == 5'. Below the code is a terminal output showing the command '\$ pytest' and the resulting test session output, which includes the platform, Python version, pytest version, root directory, and the test result for 'test\_sample.py' (F for failed) with a [100%] status. The terminal output also shows a 'FAILURES' section.

**pytest: helps you write better programs**

The `pytest` framework makes it easy to write small, readable tests, and can scale to support complex functional testing for applications and libraries.

**PyPI package name:** `pytest`

**A quick example**

```
# content of test_sample.py
def inc(x):
    return x + 1

def test_answer():
    assert inc(3) == 5
```

To execute it:

```
$ pytest
===== test session starts =====
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_sample.py F [100%]

===== FAILURES =====
```

<https://docs.pytest.org/en/stable/getting-started.html>

# PyTest Inside PyCharm

## 1) Install pytest in your project interpreter

- Open **PyCharm** → **File** → **Settings** (Windows/Linux) or **PyCharm** → **Preferences** (macOS).
- Go to **Project: <your project>** → **Python Interpreter**.
- Click **+** (or *Add Package*) and search **pytest**.
- Install.

Quick sanity check (optional).  
open PyCharm **Terminal** and run:  
`pytest --version`

## 2) Tell PyCharm to use pytest as the test runner

- Open **Settings/Preferences** → **Tools** → **Python Integrated Tools**
- In **Testing**, set **Default test runner** to **pytest**
- Apply/OK.

# PyTest Inside PyCharm

---

Another option:

open PyCharm **Terminal** and run:

```
pip install pytest
```



[https://www.youtube.com/watch?v=mt\\_LRRIGioA](https://www.youtube.com/watch?v=mt_LRRIGioA)

# PyTest Inside PyCharm



```
my_module.py ×  
1 def add(a, b): 2 usages  
2     return a + b
```

```
test_my_module.py ×  
1 from my_module import add  
2  
3 def test_add():  
4     assert add(a: 2, b: 3) == 5
```

```
===== test session starts =====  
collecting ... collected 1 item  
  
test_my_module.py::test_add PASSED [100%]  
  
===== 1 passed in 0.01s =====  
  
Process finished with exit code 0
```

**Naming rules** that make discovery work:

- Test files: test\_\*.py or \*\_test.py
- Test functions: test\_\*
- Test classes: Test\* (optional; don't define \_\_init\_\_)

[https://www.youtube.com/watch?v=mt\\_LRRIGioA](https://www.youtube.com/watch?v=mt_LRRIGioA)

`assert` is a Python keyword used to state that something **must be true**. If it is not true, Python raises an `AssertionError`.

```
x = 10
assert x > 0      # passes, nothing happens
assert x == 5     # fails -> AssertionError
```

```
Traceback (most recent call last):
  File "C:\Users\Havana\PycharmProjects\PythonCourse\Lesson8.py", line 45, in <module>
    assert x == 5      # fails -> AssertionError
    ^^^^^^^
AssertionError

Process finished with exit code 1
```

# Class-based method

---

This method is similar to the way of writing tests in unittest, except that your test class does not inherit any methods. An example is shown below:

```
class TestMathOperations:  
    def test_addition(self):  
        assert 1 + 1 == 2
```

# How to Interpret pytest Results

```
import pytest

def test_addition():
    assert 1 + 1 == 3
```

```
===== test session starts =====
platform win32 -- Python 3.10.5, pytest-8.4.1, pluggy-1.6.0
rootdir: C:\\Users\\hp\\Desktop\\Pytest
collected 1 items

test_example.py F                                     [ 50%]
                                                    [100%]

===== FAILURES =====
_____ test_addition _____

    def test_addition():
>         assert 1 + 1 == 3
E         assert (1 + 1) == 3

test_example.py:4: AssertionError

===== short test summary info =====
FAILED test_example.py::test_addition - assert (1 + 1) == 3
===== 1 failed, 1 passed in 0.13s =====
```

# How to Handle Exceptions in pytest

---

**pytest.raises** is a tool that checks if your code raises specific exceptions.

If the specified exception is raised, that test passes, confirming that the expected error occurred.

But if the specified exception is not raised, that test fails.

```
import pytest

def divide_numbers(numerator, denominator):
    return numerator / denominator

def test_divide_numbers():
    with pytest.raises(ZeroDivisionError):
        divide_numbers(10, 0)
```

# pytest Markers

---

A **marker** is just like a **label** that you can attach to a test function to categorise it. Once a test is labelled, you can instruct pytest to run only tests with certain markers.

For example, you may label some tests as "slow" if they take longer to execute and run them separately from the faster ones.

One advantage to using Markers is that it **allows** you to **run specific tests** based on categories or specific parameters, and also **skip tests** if certain conditions aren't met.

@pytest.mark.skip

@pytest.mark.skipif

@pytest.mark.xfail

# pytest Markers

---

@pytest.mark.skip:

This marker allows you to skip a test unconditionally, and can be useful when you know a test will fail due to an external issue or incomplete code.

```
@pytest.mark.skip(reason="Feature not yet implemented")  
def test_feature():  
    pass
```

# pytest Markers

---

@pytest.mark.skipif:

This marker allows you to skip a test conditionally if certain conditions are met.

```
import sys

@pytest.mark.skipif(sys.platform == "win32", reason="does not run on windows")
class TestClass:
    def test_function(self):
        "This test will not run under 'win32' platform"
```

# pytest Markers

---

@pytest.mark.xfail:

This marker is attached to tests that are expected to fail, probably due to a bug or incomplete feature. So when pytest runs such tests, it won't count it as a failure.

```
@pytest.mark.xfail(reason="division by zero not handled yet")
def test_divide_by_zero():
    assert divide(10, 0) == 0
```

# pytest Fixtures

In pytest, **fixtures** allow you to create **reusable** default data that can be shared **across multiple tests**.

By using fixtures, you can reduce code repetition, making your tests cleaner and more maintainable.

In pytest, fixtures are defined with the `@pytest.fixture` decorator

```
import pytest

@pytest.fixture
def user_data():
    return [
        {"name": "Alice", "age": 30},
        {"name": "Bob", "age": 25},
        {"name": "Charlie", "age": 35}
    ]

# Test function to check for a specific user by name and age
def test_user_exists(user_data):
    user = {"name": "Alice", "age": 30}

    # Check if the target user is in the list
    assert user in user_data

# Test average age of users
def test_average_age(user_data):
    ages = [user["age"] for user in user_data]
    avg_age = sum(ages) / len(ages)
    assert avg_age == 30
```

# Parametrization

---

Parametrization is a pytest feature that allows you to **run a test** function with **different sets of data at once**.

For example: Let's say you have a function that calculates the square of a number. To provide enough coverage while testing, you would want to test the function with zero, positive, and negative numbers.

Instead of writing separate test functions for each scenario, you can use parametrization to run a test function with different sets of data at once.

To use parametrization in pytest, we use the **@pytest.mark.parametrize** decorator

```

1  import pytest
2
3  # Function to calculate the square of a number
4  def square_numbers(num): 1 usage
5      return num * num
6
7  #Parametrize decorator to test the square function with different inputs
8  @pytest.mark.parametrize("input_value, expected_output", [
9      (2, 4),
10     (-3, 9),
11     (0, 0)
12 ])
13
14 def test_square(input_value, expected_output):
15     assert square_numbers(input_value) == expected_output

```

The different input values and expected values are listed in the **@pytest.mark.parametrize** decorator.

We're testing the `square_numbers()` function with three different input values: 2, -3, and 0.

For each value, pytest calls the `test_square()` function and compares the result of `square_numbers(input_value)` to `expected_output`.

```

1  import pytest
2
3  # Function to calculate the square of a number
4  def square_numbers(num): 1 usage
5      return num * num
6
7  #Parametrize decorator to test the square function with different inputs
8  @pytest.mark.parametrize("input_value, expected_output", [
9      (2, 4),
10     (-3, 9),
11     (0, 0)
12 ])
13
14 def test_square(input_value, expected_output):
15     assert square_numbers(input_value) == expected_output

```

```

===== test session starts =====
collecting ... collected 3 items

test_square.py::test_square[2-4] PASSED [ 33%]
test_square.py::test_square[-3-9] PASSED [ 66%]
test_square.py::test_square[0-0] PASSED [100%]

===== 3 passed in 0.04s =====

Process finished with exit code 0

```

# Prompt for Unit Testing

---

## 1. Explain the Function:

Ask the model to explain the purpose of the function.

This helps GPT model understand the context and intent, leading to more accurate test generation.

## 2. Plan Unit Tests:

Instruct GPT to plan a set of unit tests for the given function that covers:

- **Normal cases:** Typical inputs and expected outputs.
- **Edge cases:** Boundary conditions, empty inputs, max/min values, etc.
- **Error handling/exceptions:** Invalid inputs or exceptional scenarios.
- **Specific requirements:** Particular conditions or behaviors you want to test.

## 3. Generate Unit Tests:

Based on the plan, instruct GPT to write the actual unit tests. Specify the testing framework (in our case Pytest) and any desired conventions (e.g., AAA principle - Arrange, Act, Assert).

# Try it Yourself

---

**Solve the questions in the notebook**



# GitHub

The screenshot displays the GitHub web interface for a user named 'HavanaLab'. The top navigation bar includes links for Overview, Repositories (17), Projects (2), Packages, Teams, People (2), Insights, and Settings. A search bar is located on the right. The left sidebar shows filters for All, Public, Private, Sources, Forks, Archived, and Templates. The main content area, titled 'All', shows a list of 17 repositories. Each repository entry includes its name, privacy status, language, star/fork/watch counts, and the last update time. Some repositories also show a small green line graph representing commit activity.

| Repository Name         | Privacy | Language         | Stars | Forks | Watches | Last Updated            |
|-------------------------|---------|------------------|-------|-------|---------|-------------------------|
| AnimalBreedsDetection   | Private |                  | 0     | 0     | 0       | Updated 3 days ago      |
| ForecastingCruiseRoutes | Private | Python           | 0     | 1     | 0       | Updated 2 weeks ago     |
| clique_aaai_2026        | Public  | Jupyter Notebook | 0     | 0     | 0       | Updated 3 weeks ago     |
| Roots                   | Private | Python           | 0     | 0     | 0       | Updated last month      |
| GeneralOptGNNforCO      | Private | Jupyter Notebook | 0     | 0     | 0       | Updated last month      |
| CliqueEvaluation        | Private | Jupyter Notebook | 0     | 0     | 0       | Updated on Oct 27       |
| DeepMindMaxClique       | Private | Python           | 0     | 0     | 0       | Updated on Aug 15       |
| graph_coloring          | Public  | Python           | 0     | 0     | 0       | Updated on Jan 21       |
| GCP_project             | Public  | Python           | 1     | 0     | 0       | Updated on Jan 18       |
| FeatureSelection        | Private |                  | 0     | 0     | 0       | Updated on Oct 26, 2024 |

# GitHub

The screenshot shows the GitHub interface for a repository named 'ForecastingCruiseRoutes' by user 'liorkh'. The repository is private and has 43 branches, 0 tags, and 171 commits. The file list includes 'data', 'data\_algorithms', 'data\_creation', 'data\_explore', 'data\_perprocessing', '.gitignore', and 'main.py'. The 'About' section is empty. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows Python at 100.0%. The 'Suggested workflows' section shows a workflow for 'SLSA Generic generator'. The 'Add a README' section is visible at the bottom.

HavanaLab / ForecastingCruiseRoutes

Code Issues Pull requests Actions Projects Security Insights Settings

ForecastingCruiseRoutes Private

master 43 Branches 0 Tags

Go to file Add file Code

liorkh created val+train 75c46b4 · 4 months ago 171 Commits

|                    |                                            |               |
|--------------------|--------------------------------------------|---------------|
| data               | Change distance to haversine               | 10 months ago |
| data_algorithms    | fixed bug in loop for training train + val | 5 months ago  |
| data_creation      | removed redundant data                     | 5 months ago  |
| data_explore       | dump to pickle                             | 5 months ago  |
| data_perprocessing | created val+train                          | 4 months ago  |
| .gitignore         | changed parameters at tun_model_k_fold     | 5 months ago  |
| main.py            | Creating dictionary with data              | last year     |

README

Add a README

Help people interested in this repository understand your project by adding a README.

Add a README

About

No description, website, or topics provided.

Activity

Custom properties

1 star

1 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

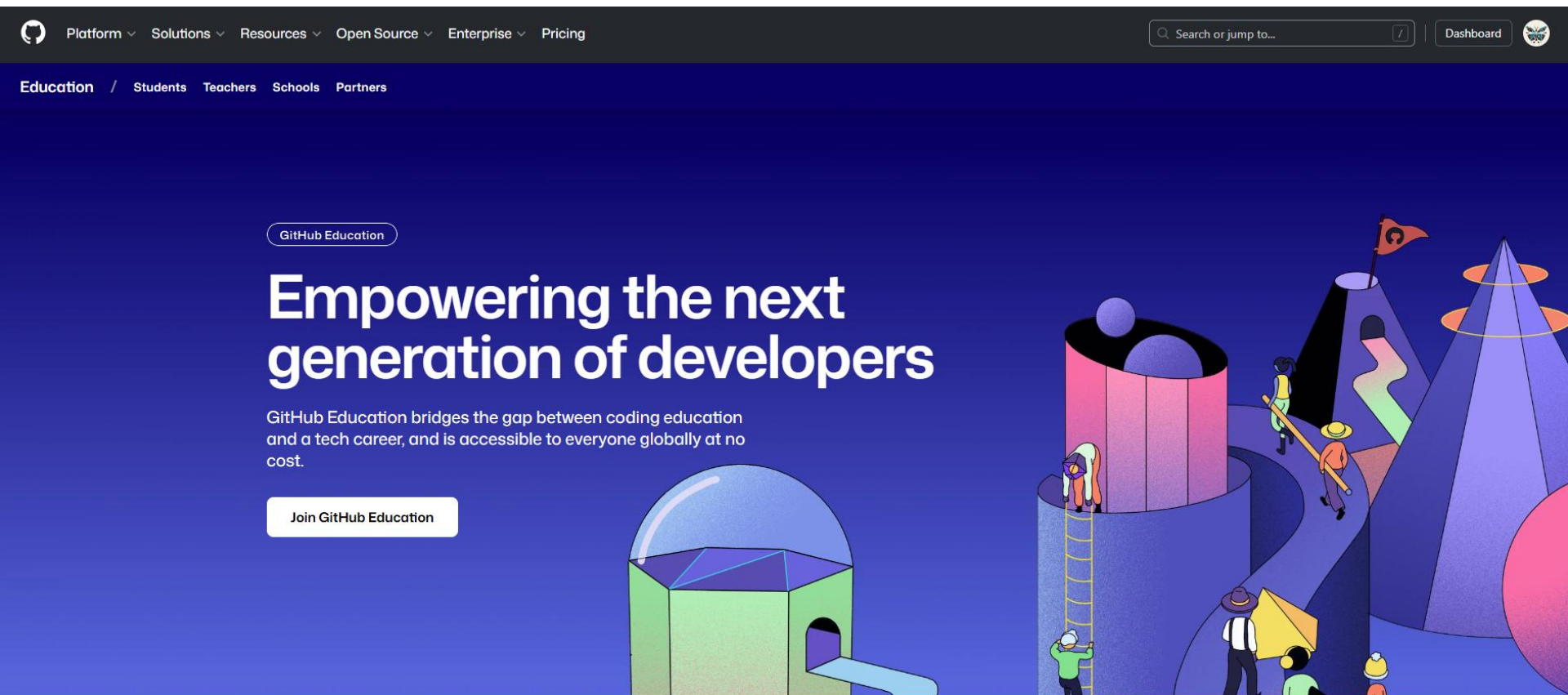
Python 100.0%

Suggested workflows

Based on your tech stack

SLSA Generic generator Configure

# GitHub Education



# Homework for Next Week

---

1. Open a GitHub account.
2. And sign up to GitHub Education as a student.

